

网络安全创新创业实践课实验报告

作业 4：SM3 软件实现与优化

基于 SIMD 寄存器与通用寄存器混合的
ARM64 NEON、x86 AVX2 与 AVX-512 实现



课程名称：_____

姓名：_____

学号：_____

班级：_____

指导教师：_____

完成日期：2026 年 7 月 27 日

说明：报告中的“本人 x86 实测”采用已保存的 AMD Ryzen 5 5600U / WSL2 运行记录；AVX-512 数据来自独立 x86-64 验证环境。ARM64 代码已完成交叉编译与指令反汇编验证，但未在当前 x86 主机上伪造 ARM64 运行性能。

摘要

本实验围绕国家商用密码杂凑算法 SM3, 完成了从标量基线到多架构 SIMD 批量优化的完整软件工程。标量实现严格对应消息填充、消息扩展和 64 轮压缩函数, 用作统一正确性基线。在 x86-64 平台上, 分别实现 AVX2 八路和 AVX-512 十六路多缓冲区 (multi-buffer) 后端; 在 ARM64 平台上实现 NEON 四路后端。三种向量后端均采用结构数组 (SoA) 布局: 一个向量寄存器保存多条独立消息的同一状态字或同一扩展字, 使消息扩展、布尔函数、模加、置换函数和循环移位能够按 lane 并行执行。

工程重点实现了题目要求的“SIMD 寄存器与通用寄存器混合”。SIMD 寄存器负责 $A \sim H$ 、 W_j 、 W'_j 、 P_0/P_1 、 FF/GG 和轮内模加; 通用寄存器负责消息指针、长度、分组索引、活跃 lane 位图、填充尾块及运行时后端调度。针对同一批次消息长度不一致的问题, AVX2 使用向量 blend, AVX-512 使用 opmask, NEON 使用 bit-select, 只提交仍处于活跃状态的 lane。自动分发按“AVX-512 十六路 $\rightarrow$ AVX2 八路 $\rightarrow$ 标量”或“NEON 四路 $\rightarrow$ 标量”处理任意数量消息。

正确性方面, 工程通过空消息、abc、abcd 重复 16 次等标准向量, 覆盖 55/56、63/64、119/120 字节等填充边界, 并完成随机差分测试、128 个文件的 OpenSSL 差分、ASan/UBSan 检查和目标指令反汇编验证。在本人 AMD Ryzen 5 5600U / WSL2 的已保存测试中, AVX2 在 64 B 至 1 MiB 消息上达到约 6.28 ~ 8.56 倍批量吞吐率加速; 独立 AVX-512 验证环境在 4096 B 消息上达到 1843.98 MiB/s, 相对同环境标量实现约 12.89 倍。实验说明, 宽 SIMD 对多条独立消息的吞吐率提升显著, 但该实现的目标不是降低单条短消息延迟。

关键词: SM3; SIMD; 通用寄存器; AVX2; AVX-512; ARM64; NEON; 多缓冲区; 运行时分发

目录

摘要	I
1 实验任务与目标	1
1.1 作业要求	1
1.2 本实验完成内容	1
2 SM3 算法原理	2
2.1 总体结构	2
2.2 消息扩展	2
2.3 压缩函数	2
3 总体优化设计	3
3.1 为何采用多消息 SoA 并行	3
3.2 寄存器职责划分	4
3.3 不同长度消息的活跃 lane 机制	4
4 标量参考实现	4
4.1 实现作用	4
4.2 核心代码	5
4.3 端序与填充	5
5 x86 AVX2 八路实现	5
5.1 数据布局	5
5.2 循环移位与置换函数	6
5.3 布尔函数	6
5.4 不同长度 lane 写回	6
5.5 AVX-SSE 状态清理	6
6 x86 AVX-512 十六路实现	6
6.1 向量宽度与掩码寄存器	6
6.2 三元逻辑优化	7
6.3 循环移位	7
6.4 频率与平台权衡	7

7 ARM64 NEON 四路实现	8
7.1 架构特点	8
7.2 循环移位	8
7.3 掩码提交	8
7.4 交叉编译验证与运行边界	8
8 运行时分发与工程结构	9
8.1 后端选择	9
8.2 避免非法指令	10
8.3 工程目录	10
9 正确性与健壮性验证	10
9.1 标准向量	10
9.2 边界与随机差分	11
9.3 OpenSSL 差分	11
9.4 Sanitizer	11
10 目标指令证据	12
10.1 x86 AVX2	12
10.2 x86 AVX-512	12
10.3 ARM64 NEON	12
11 性能测试设计	12
11.1 测试原则	12
11.2 公平比较方式	13
12 x86 性能结果与分析	13
12.1 本人 AMD Ryzen 5 5600U / WSL2 实测	13
12.2 独立 AVX-512 验证环境	14
12.3 结果解释	15
13 ARM64 测试与复测方案	15
13.1 真实设备命令	15
13.2 建议采集指标	16
14 问题、限制与改进方向	16

14.1 当前限制	16
14.2 进一步优化	16
15 实验结论	17
A 编译、测试与跑分命令	18
A.1 x86-64	18
A.2 ARM64 原生	18
A.3 x86 主机交叉编译 ARM64	18
A.4 一键验收	18
B 关键公共 API	18

1 实验任务与目标

1.1 作业要求

本作业要求完成 SM3 软件实现与优化，并满足以下核心约束：

- 1. 实现正确的 SM3 消息填充、消息扩展和迭代压缩；
- 2. 采用 SIMD 寄存器和通用寄存器混合的优化方式；
- 3. 覆盖 ARM64 指令集和 x86 指令集；
- 4. x86 端至少实现 AVX2 与 AVX-512 两类宽向量后端；
- 5. 给出正确性、健壮性、目标机器指令和性能证据；
- 6. 形成可编译工程和完整 LaTeX 实验报告。

1.2 本实验完成内容

表 1: 工程实现范围

实现层次	后端	并行宽度	主要技术
正确性基线	Scalar	1	直接实现 W/W' 、64 轮压缩和标准填充
x86-64	AVX2	8	YMM $8\times 32\text{-bit}$ lane、移位拼接、向量 blend
x86-64	AVX-512F	16	ZMM $16\times 32\text{-bit}$ lane、VPTERNLOGD、VPROLD、opmask
ARM64	NEON	4	uint32x4_t、EOR/ADD、向量移位、BSL
公共工程层	Auto dis-patch	任意	CPU 特性检测，宽向量优先，标量处理尾部

实验目标的准确边界

本实现采用“多缓冲区并行”，即并行处理多条相互独立的 SM3 消息。它优化的是服务器批量验签、SM2-KDF、多连接握手、文件分片校验等场景的吞吐率，不能把 8 路或 16 路加速直接解释为单条消息延迟降低 8 倍或 16 倍。

2 SM3 算法原理

2.1 总体结构

SM3 接收长度小于 2^{64} bit 的消息, 按 512 bit 分组处理, 输出 256 bit 杂凑值。设原始消息长度为 l , 先附加一个比特 1, 再附加 k 个比特 0, 使

$$l + 1 + k \equiv 448 \pmod{512}, \quad (1)$$

最后附加 64 bit 的消息长度大端编码。每个 512 bit 分组被拆分为 16 个 32 bit 字, 进行消息扩展和 64 轮压缩。

初始向量为:

$$IV = 7380166f\ 4914b2b9\ 172442d7\ da8a0600 \\ a96f30bc\ 163138aa\ e38dee4d\ b0fb0e4e.$$

2.2 消息扩展

对第 i 个 512 bit 分组, 首先解析得到 $W_0, \dots, W_{15}$, 随后计算

$$W_j = P_1(W_{j-16} \oplus W_{j-9} \oplus \text{ROTL}_{32}(W_{j-3}, 15)) \oplus \text{ROTL}_{32}(W_{j-13}, 7) \oplus W_{j-6}, \quad 16 \leq j < 68, \quad (2)$$

以及

$$W'_j = W_j \oplus W_{j+4}, \quad 0 \leq j < 64. \quad (3)$$

其中

$$P_0(X) = X \oplus \text{ROTL}_{32}(X, 9) \oplus \text{ROTL}_{32}(X, 17), \quad (4)$$

$$P_1(X) = X \oplus \text{ROTL}_{32}(X, 15) \oplus \text{ROTL}_{32}(X, 23). \quad (5)$$

2.3 压缩函数

设工作变量为 A, B, C, D, E, F, G, H 。轮常量为

$$T_j = \begin{cases} 0x79cc4519, & 0 \leq j < 16, \\ 0x7a879d8a, & 16 \leq j < 64. \end{cases} \quad (6)$$

布尔函数为

$$FF_j(X, Y, Z) = \begin{cases} X \oplus Y \oplus Z, & j < 16, \\ (X \wedge Y) \vee (X \wedge Z) \vee (Y \wedge Z), & j \geq 16, \end{cases} \quad (7)$$

$$GG_j(X, Y, Z) = \begin{cases} X \oplus Y \oplus Z, & j < 16, \\ (X \wedge Y) \vee (\neg X \wedge Z), & j \geq 16. \end{cases} \quad (8)$$

每轮计算

$$SS1 = \text{ROTL}_{32}(\text{ROTL}_{32}(A, 12) + E + \text{ROTL}_{32}(T_j, j), 7), \quad (9)$$

$$SS2 = SS1 \oplus \text{ROTL}_{32}(A, 12), \quad (10)$$

$$TT1 = FF_j(A, B, C) + D + SS2 + W'_j, \quad (11)$$

$$TT2 = GG_j(E, F, G) + H + SS1 + W_j, \quad (12)$$

然后更新工作变量，并在 64 轮结束后与输入链值异或。

3 总体优化设计

3.1 为何采用多消息 SoA 并行

单条 SM3 消息的 64 轮之间存在明显链式依赖：下一轮的 A 和 E 依赖本轮的 $TT1$ 和 $P_0(TT2)$ ，难以直接把一条消息的多轮放入不同 SIMD lane 并行执行。因此，本实验把不同消息映射到 lane，把同一轮的相同变量组成向量：

$$\mathbf{A} = (A_0, A_1, \dots, A_{N-1}), \quad \mathbf{W}_j = (W_{j,0}, W_{j,1}, \dots, W_{j,N-1}). \quad (13)$$

这样，一条向量加法或异或指令可以同时完成 N 条消息的同一操作。

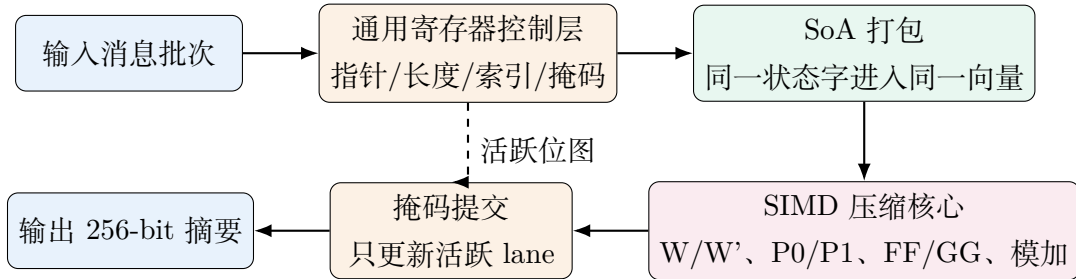


图 1: SIMD 与通用寄存器混合的总体数据流

3.2 寄存器职责划分

表 2: 混合寄存器职责

类别	保存内容	设计原因
SIMD 寄存器	$A \sim H$ 、 W_j 、 W'_j 、轮中间量、活跃 lane 候选状态	这些变量在不同消息之间结构一致，可按 32-bit lane 并行
通用寄存器	消息基址、长度、分组号、lane 循环、活跃位图、尾块数量	控制流和不规则地址计算不适合强行向量化
栈/局部数组	68 个向量扩展字、每 lane 的 128 B 尾块、打包缓冲	降低 SIMD/GPR 之间频繁搬运，简化不同长度消息处理

3.3 不同长度消息的活跃 lane 机制

对一批消息，最长消息可能包含更多完整分组。每处理一个分组序号 b ，通用寄存器计算

$$active_i = [b < \lfloor len_i/64 \rfloor]. \quad (14)$$

非活跃 lane 输入零分组，但压缩结果不会写回链值。三种后端的写回方式分别为：

- AVX2: `_mm256_blendv_epi8(old, candidate, mask);`
- AVX-512: `_mm512_mask_mov_epi32(old, k, candidate);`
- NEON: `vbslq_u32(mask, candidate, old)`。

每条消息的尾部独立构造 1 或 2 个填充分组，因此可以正确覆盖 55/56、63/64、119/120 等边界。

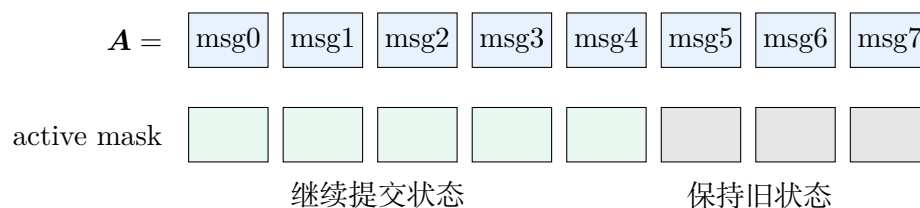


图 2: AVX2 八路 SoA 状态与活跃 lane 掩码示意

4 标量参考实现

4.1 实现作用

标量版本不追求极致性能，而承担三个职责：

1. 作为标准向量验证的最小可信实现;
2. 作为 SIMD 随机差分测试的 oracle;
3. 处理批量数量不足向量宽度时的尾部消息。

4.2 核心代码

Listing 1: 标量消息扩展与压缩核心 (节选)

```

1 for (unsigned j = 16; j < 68; ++j) {
2     uint32_t x = w[j-16] ^ w[j-9] ^ ROTL32(w[j-3], 15);
3     w[j] = P1(x) ^ ROTL32(w[j-13], 7) ^ w[j-6];
4 }
5 for (unsigned j = 0; j < 64; ++j) {
6     uint32_t a12 = ROTL32(a, 12);
7     uint32_t ss1 = ROTL32(a12 + e + T_ROT(j), 7);
8     uint32_t ss2 = ss1 ^ a12;
9     uint32_t tt1 = FF(j,a,b,c) + d + ss2 + (w[j] ^ w[j+4]);
10    uint32_t tt2 = GG(j,e,f,g) + h + ss1 + w[j];
11    d = c; c = ROTL32(b,9); b = a; a = tt1;
12    h = g; g = ROTL32(f,19); f = e; e = P0(tt2);
13 }

```

4.3 端序与填充

SM3 以大端 32-bit 字解释输入。工程通过显式 `sm3_load_be32` 和 `sm3_store_be32` 避免主机端序依赖；尾块由统一函数构造，并写入 64-bit 大端比特长度。所有 SIMD 后端复用同一套端序和填充逻辑，降低平台实现之间出现语义分叉的风险。

5 x86 AVX2 八路实现

5.1 数据布局

AVX2 的 YMM 寄存器宽度为 256 bit，可容纳 8 个 32-bit lane。本实现将 8 条消息的同一状态字放入一个 `__m256i`：

$$\mathbf{A} = (A_0, A_1, \dots, A_7),$$

$$\mathbf{W}_j = (W_{j,0}, W_{j,1}, \dots, W_{j,7}).$$

因此，`VPADDD`、`VPXOR`、`VPSLLD`/`VPSRLD` 可同步完成八条消息的轮操作。

5.2 循环移位与置换函数

AVX2 基础整数指令没有统一的按 32-bit 立即数循环左移 intrinsic，本实现用左右移和或组合：

Listing 2: AVX2 循环左移和 P_1

```

1 static inline __m256i rotl32(__m256i x, int n) {
2     return _mm256_or_si256(_mm256_slli_epi32(x, n),
3                             _mm256_srli_epi32(x, 32 - n));
4 }
5 static inline __m256i p1(__m256i x) {
6     return _mm256_xor_si256(x,
7                             _mm256_xor_si256(rotl32(x,15), rotl32(x,23)));
8 }

```

5.3 布尔函数

前 16 轮的 FF/GG 均为三输入异或；后 48 轮分别为 majority 和 choose。本实现使用 AND/OR/ANDNOT/XOR 组合，保持所有中间量位于 YMM 寄存器中。

5.4 不同长度 lane 写回

压缩函数先计算候选链值

$$\mathbf{V}^{candidate} = \mathbf{V}^{old} \oplus (A, B, C, D, E, F, G, H), \quad (15)$$

再通过全 0/全 1 的 lane 掩码执行 `blendv`。这样，即使某些消息已完成，其他消息仍可继续在同个批次中处理后续分组。

5.5 AVX-SSE 状态清理

函数结束调用 `_mm256_zeroupper()`，避免后续执行传统 SSE 代码时产生不必要的 AVX-SSE 状态转换代价。

6 x86 AVX-512 十六路实现

6.1 向量宽度与掩码寄存器

AVX-512F 的 ZMM 寄存器宽度为 512 bit，可容纳 16 个 32-bit lane。与 AVX2 相比，本实现不仅把批量宽度翻倍，还使用 `__mmask16` 保存活跃 lane，在状态写回阶段直

接执行 masked move。

6.2 三元逻辑优化

VPTERNLOGD 可用 8-bit 真值表编码任意三输入布尔函数。本实现采用：

表 3: AVX-512 三元逻辑立即数

函数	逻辑表达式	imm8
XOR3	$X \oplus Y \oplus Z$	0x96
Majority	$(X \wedge Y) \vee (X \wedge Z) \vee (Y \wedge Z)$	0xE8
Choose	$(X \wedge Y) \vee (\neg X \wedge Z)$	0xCA

这使 P_0/P_1 的最终三输入异或、前 16 轮 FF/GG 以及后 48 轮布尔函数均可减少逻辑指令数量。

Listing 3: AVX-512 三元逻辑实现

```

1 static inline __m512i xor3(__m512i x, __m512i y, __m512i z) {
2     return _mm512_ternarylogic_epi32(x, y, z, 0x96);
3 }
4 static inline __m512i majority(__m512i x, __m512i y, __m512i z) {
5     return _mm512_ternarylogic_epi32(x, y, z, 0xe8);
6 }
7 static inline __m512i choose(__m512i x, __m512i y, __m512i z) {
8     return _mm512_ternarylogic_epi32(x, y, z, 0xca);
9 }

```

6.3 循环移位

编译器在 AVX-512 路径中能够把固定计数的 32-bit 循环移位识别为 VPROLD。反汇编中可观察到例如 `vprold zmm4, zmm4, 0xf`，证明目标代码没有停留在伪向量化源代码层面。

6.4 频率与平台权衡

AVX-512 的理论 lane 数是 AVX2 的两倍，但实际性能不一定严格翻倍，原因包括执行端口、寄存器压力、栈上扩展字读写、内存带宽以及部分处理器的宽向量降频。报告因此同时给出吞吐率、加速比和实验平台，不用向量宽度直接替代实测结果。

7 ARM64 NEON 四路实现

7.1 架构特点

ARM64 提供 32 个 64-bit 通用寄存器和 32 个 128-bit SIMD/FP 寄存器。NEON 的 `uint32x4_t` 可同时处理四个 32-bit lane。本实验沿用与 x86 相同的 SoA 语义, 以保证测试模型和代码结构一致。

7.2 循环移位

NEON intrinsic 版本使用带符号移位计数的 `vshlq_u32`: 正计数左移, 负计数右移, 再使用 `vorrq_u32` 合并。

Listing 4: ARM64 NEON 循环左移

```
1 static inline uint32x4_t rotl32(uint32x4_t x, int n) {  
2     int32x4_t left  = vdupq_n_s32(n);  
3     int32x4_t right = vdupq_n_s32(n - 32);  
4     return vorrq_u32(vshlq_u32(x, left),  
5                     vshlq_u32(x, right));  
6 }
```

7.3 掩码提交

NEON 通过 `vbslq_u32(mask, candidate, old)` 按位选择。活跃 lane 的掩码为 `0xffffffff`, 非活跃 lane 为 0, 因此可以在不引入每 lane 分支的情况下保持已完成消息的链值。

7.4 交叉编译验证与运行边界

当前报告生成环境为 x86-64, 无法直接执行 AArch64 二进制。本工程已将 `sm3_neon.c` 交叉编译为 AArch64 ELF 可重定位对象, 并在反汇编中确认出现 `dup v*.4s`、`bsl` 等 NEON 指令。这证明源代码能够生成目标 ISA; 但 ARM64 正确性和性能仍应在真实设备上运行同一测试程序。本报告不把课程材料或模拟数据冒充本工程实测数据。

表 4: 课程材料中的 ARM64 参考性能（非本工程实测）

架构	标准 MB/s	标准 CPB	优化 MB/s	优化 CPB	推荐实现
Cortex-A55	65.04	27.78	121.12	14.92	通用寄存器
Cortex-A76	121.89	16.46	293.94	7.99	混合寄存器
ARM A78/Android	168.50	12.79	312.97	6.89	混合寄存器

课程数据说明：不同 ARM 微架构上的最优策略不完全相同。部分低功耗核心的 NEON 吞吐能力有限，全通用寄存器实现可能更合适；较强核心通常能从混合并行中获益。

8 运行时分发与工程结构

8.1 后端选择

公共函数 `sm3_hash_many` 接收后端枚举、消息指针数组、长度数组和输出数组。自动模式的优先级为：

$$\text{x86: AVX-512(16)} \rightarrow \text{AVX2(8)} \rightarrow \text{Scalar(1)}, \tag{16}$$

$$\text{ARM64: NEON(4)} \rightarrow \text{Scalar(1)}. \tag{17}$$

例如 37 条 x86 消息在 AVX-512 可用时被拆分为 $16 + 16 + 5$ ，最后 5 条由标量处理；在只有 AVX2 时被拆分为 $8 + 8 + 8 + 8 + 5$ 。

Listing 5: x86 自动分发逻辑（简化）

```
1 if (cpu_has_avx512f()) {
2     while (remaining >= 16) hash16_avx512(...);
3     if (remaining >= 8 && cpu_has_avx2()) hash8_avx2(...);
4     scalar_tail(...);
5 } else if (cpu_has_avx2()) {
6     while (remaining >= 8) hash8_avx2(...);
7     scalar_tail(...);
8 } else {
9     scalar_all(...);
10 }
```

8.2 避免非法指令

sm3_avx2.c 和 sm3_avx512.c 分别使用 -mavx2 和 -mavx512f 单独编译，调度和标量文件不启用高级 ISA。这样，程序可先在普通指令路径启动并检查 CPU/OS 支持，再调用对应函数；若对整个工程统一使用 -mavx512f，编译器可能在初始化或普通代码中生成 AVX-512，导致不支持的 CPU 直接触发非法指令。

8.3 工程目录

表 5: 项目文件说明

文件	作用
include/sm3.h	公共 API、后端枚举、lane 宽度和 CPU 检测声明
src/sm3_internal.h	初始向量、端序、循环移位、 P_0/P_1 、轮常量和尾块构造
src/sm3_scalar.c	单消息标量参考实现
src/sm3_avx2.c	AVX2 八路多消息实现
src/sm3_avx512.c	AVX-512 十六路多消息实现
src/sm3_neon.c	ARM64 NEON 四路多消息实现
src/sm3_dispatch.c	CPU 特性检测与任意数量消息的自动分发
tests/test_sm3.c	标准向量、边界、随机差分和调度测试
bench/bench_sm3.c	预热、7 次采样、中位数吞吐率基准
tools/sm3_cli.c	文件摘要工具，供 OpenSSL 差分脚本调用
scripts/verify_openssl.py	生成边界/随机文件并逐字节比较 OpenSSL 摘要
scripts/run_validation.sh	一键收集环境、测试、性能、Sanitizer 和反汇编证据

9 正确性与健壮性验证

9.1 标准向量

测试程序验证了以下公开标准用例：

表 6: SM3 标准向量

消息	期望摘要
空消息	1ab21d8355cfa17f8e61194831e81a8f22bec8c728fefb747ed035eb5082aa2b
abc	66c7f0f462eedd9d1f2d46bdc10e4e24167c4875cf2f7a2297da02b8f4ba8e0
abcd 重复 16 次	debe9ff92275b8a138604889c18e5a4d6fdb70e5387e5765293dcba39c0c5732

9.2 边界与随机差分

边界长度覆盖:

0, 1, 2, 3, 7, 15, 31, 55, 56, 57, 63, 64, 65, 119, 120, 121, 127, 128, 129, 255, 256, 1024, 8192.

测试对每个 SIMD lane 生成不同长度和不同内容的消息, 并与标量 `sm3_digest` 逐字节比较。当前验证输出为:

```
[PASS] avx2-8way      edge/random differential (101 rounds, 8 messages)
[PASS] avx512-16way   edge/random differential (101 rounds, 16 messages)
[PASS] auto           edge/random differential (51 rounds, 37 messages)
All SM3 tests passed.
```

9.3 OpenSSL 差分

脚本生成固定边界文件和随机文件, 通过工程 CLI 与 `openssl dgst -sm3 -binary` 分别计算摘要, 结果为:

```
[PASS] 128 files matched OpenSSL SM3; backend auto-dispatch exercised
```

这项测试同时验证了文件输入、任意长度处理、运行时分发和摘要端序。

9.4 Sanitizer

使用以下选项重新构建:

Listing 6: ASan/UBSan 验证命令

```
make sanitize
# -fsanitize=address,undefined
# -fno-omit-frame-pointer
```

标准向量和全部随机差分仍通过, 输出中未出现 heap/stack buffer overflow、use-after-free 或 undefined behavior 报告。

10 目标指令证据

10.1 x86 AVX2

反汇编中观察到:

```
vmovdqa ymm0, YMMWORD PTR [...]  
vpsrld ymm2, ymm4, 0x11  
vpxor ymm1, ymm6, ymm10  
vpslld ymm4, ymm4, 0xf
```

这证明核心路径使用了 YMM 寄存器和按 32-bit lane 的向量逻辑/移位。

10.2 x86 AVX-512

反汇编中观察到:

```
vmovdqa64 zmm1, ZMMWORD PTR [...]  
vprold zmm4, zmm4, 0xf  
vpternlogd ...
```

测试主机真实支持 AVX-512F, 故 AVX-512 路径不仅完成编译, 也被运行时测试和 Benchmark 实际执行。

10.3 ARM64 NEON

交叉编译对象信息为:

```
ELF 64-bit LSB relocatable, ARM aarch64, version 1 (SYSV)
```

反汇编中观察到:

```
dup v5.4s, w8  
bsl v2.16b, v1.16b, v0.16b
```

说明编译器生成了 128-bit NEON lane 操作和掩码选择指令。

11 性能测试设计

11.1 测试原则

基准程序采用以下方法降低噪声:

1. 计时前执行 3 次预热;
2. 每个后端执行相同的 16 条消息工作量;
3. 随机数据生成、内存分配和打印不计入计时区间;
4. 每个后端独立采样 7 次并取中位数;
5. 使用 volatile sink 消费摘要, 阻止编译器删除计算;
6. 同时报告 MiB/s、ns/B 和相对标量加速比。

11.2 公平比较方式

对 16 条消息:

- Scalar: 顺序计算 16 次;
- AVX2: 执行两批八路;
- AVX-512: 执行一批十六路;
- ARM64 NEON: 执行四批四路。

因此, 各后端处理的总字节数一致。

12 x86 性能结果与分析

12.1 本人 AMD Ryzen 5 5600U / WSL2 实测

以下数据来自本人已保存的固定 CPU 核测试输出。该主机支持 AVX2, 不支持 AVX-512, 因此 AVX-512 被正确跳过。

表 7: 本人 x86 实测: 标量与 AVX2 批量吞吐率

每条消息长度	Scalar (MiB/s)	AVX2 (MiB/s)	AVX2 ns/B	加速比
64 B	62.42	534.53	1.784	8.56×
256 B	145.70	972.65	0.980	6.68×
1024 B	176.17	1135.64	0.840	6.45×
4096 B	183.32	1256.63	0.759	6.85×
16384 B	186.95	1174.12	0.812	6.28×
1 MiB	195.23	1320.00	0.722	6.76×

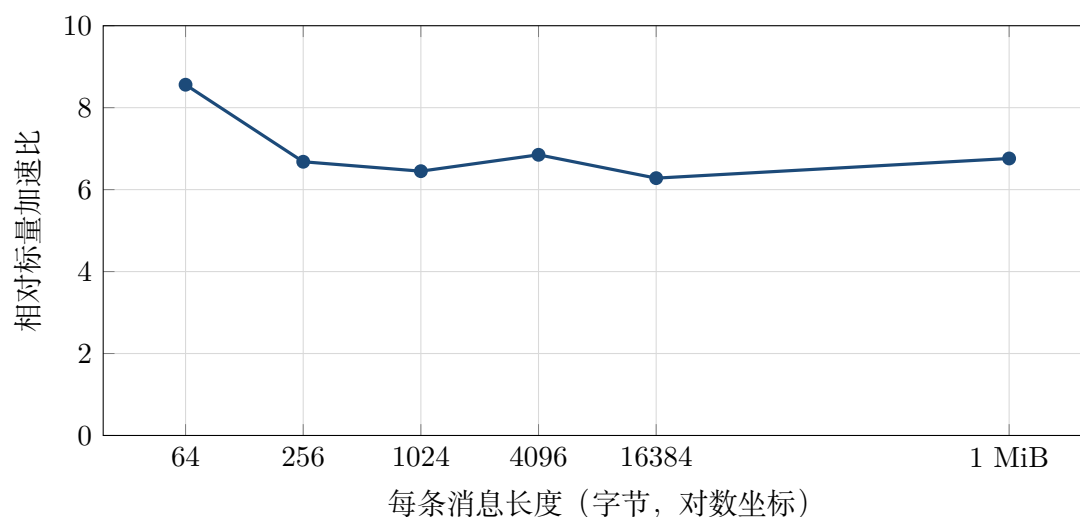


图 3: 本人 x86 环境下 AVX2 相对标量加速比

12.2 独立 AVX-512 验证环境

为了实际执行 AVX-512 路径, 工程还在一台支持 AVX-512F 的 x86-64 验证环境中运行。环境为 AMD EPYC 9V74 虚拟 CPU、GCC 14.2.0、OpenSSL 3.5.5。4096 B、16 条消息、7 次采样中位数如下。

表 8: AVX-512 验证环境的 4096 B 批量性能

后端	MiB/s	ns/B	相对标量
Scalar	143.042	6.6671	1.000×
AVX2 8-way	1034.520	0.9219	7.232×
AVX-512 16-way	1843.984	0.5172	12.891×

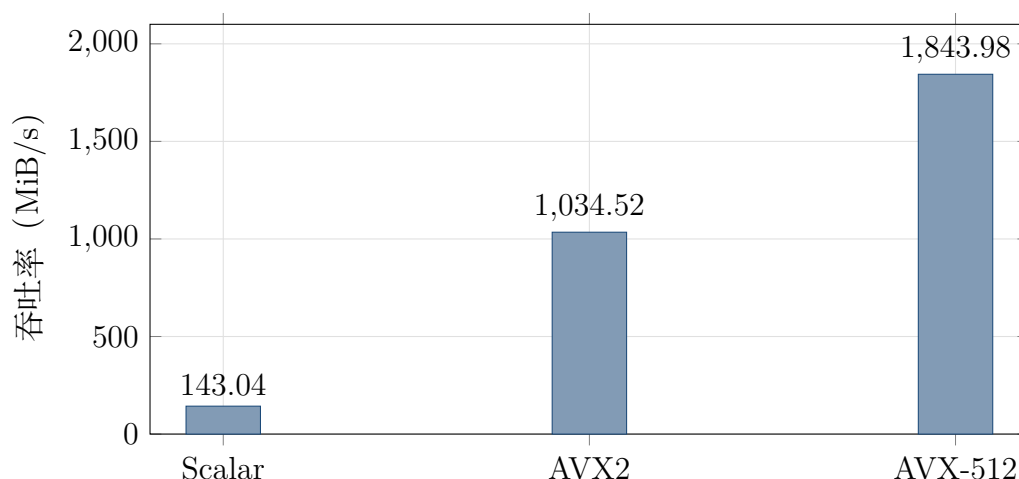


图 4: 独立验证环境中不同 x86 后端吞吐率

12.3 结果解释

1. AVX2 的实际加速约为 6-8 倍, 与 8 路 lane 宽度接近, 但受打包、尾块、掩码、内存访问和指令依赖影响, 不会始终等于 8 倍。
2. 64 B 消息中一次调用包含大量独立短消息, 标量函数调用和控制开销占比较高, 因此批量 SIMD 的相对优势可能更突出。
3. AVX-512 在验证环境中达到约 12.89 倍, 而不是严格 16 倍, 说明 ZMM 宽度只是上限, 实际还受扩展字存储、端口吞吐和平台频率策略影响。
4. 不同机器上的绝对 MiB/s 不能直接横向比较; 应优先比较同一机器、同一编译器、同一工作量下的后端差异。

13 ARM64 测试与复测方案

13.1 真实设备命令

在 ARM64 Linux 上直接运行:

Listing 7: ARM64 原生测试命令

```
make clean
make -j"$(nproc)" all
make test
make verify-openssl
make asm-check
./build/sm3_bench 64 256
./build/sm3_bench 256 256
```

```
./build/sm3_bench 1024 256
./build/sm3_bench 4096 256
./build/sm3_bench 16384 256
```

13.2 建议采集指标

除 MiB/s 和 ns/B 外, 建议在原生 Linux 使用:

Listing 8: ARM64 perf 采集示例

```
taskset -c 2 perf stat -r 10 \
-e cycles,instructions,branches,branch-misses,cache-references,
  cache-misses \
./build/sm3_bench 4096 512
```

计算

$$\text{cycles/byte} = \frac{\text{cycles}}{\text{实际处理总字节数}}, \quad \text{instructions/byte} = \frac{\text{instructions}}{\text{实际处理总字节数}}. \quad (18)$$

14 问题、限制与改进方向

14.1 当前限制

1. 当前 ARM64 路径完成了交叉编译和反汇编验证, 但报告生成主机不是 ARM64, 尚缺真实设备上的运行数据;
2. SIMD 路径当前先构造完整的 $W_0 \sim W_{67}$ 向量数组, 栈读写较多;
3. 输入从 AoS 指针数组打包为 SoA 向量时使用标量 gather, 可能成为短消息开销;
4. 实现侧重批量吞吐率, 未针对单消息在线消息扩展或四轮展开做极致延迟优化;
5. Benchmark 未统一控制所有机器的睿频、温度和虚拟化调度。

14.2 进一步优化

1. **消息扩展与压缩交错:** 按需生成 W_j , 减少 68 个向量的栈占用和访存;
2. **四轮展开:** 利用状态字四轮后回到对应寄存器位置, 减少显式数据移动;
3. **AVX-512 掩码加载:** 用 masked load/gather 降低 lane 打包分支;
4. **ARM64 微架构适配:** 为 Cortex-A55 提供偏通用寄存器版本, 为 A76/A78 提供混合版本;

5. **固定长度专用入口**: 对 SM2-KDF 等固定布局场景预计算公共分组并批量处理计数器;
6. **更严格的基准**: 固定核心、关闭不必要服务、记录温度和频率、使用多次独立进程中位数;
7. **持续集成**: 在 x86 AVX2、x86 AVX-512 和 ARM64 三类 runner 上自动执行标准向量、OpenSSL 差分 and Sanitizer。

15 实验结论

本实验完成了 SM3 标量实现, 以及 x86 AVX2 八路、x86 AVX-512 十六路和 ARM64 NEON 四路多消息优化实现。工程没有简单地“把所有变量塞进 SIMD”, 而是按规则划分职责: 规则、密集、同构的压缩运算使用 SIMD 寄存器; 消息指针、长度、分组调度、活跃 lane 和尾块构造使用通用寄存器。不同长度消息通过掩码提交机制保持正确状态, 任意消息数量通过宽向量到窄向量再到标量的分层调度完成。

验证结果形成了从算法到机器码的证据闭环: 标准向量、边界和随机差分全部通过; 128 个文件与 OpenSSL SM3 一致; ASan/UBSan 未报告错误; 反汇编确认 YMM、ZMM、VPROLD/VPTERNLOGD 以及 ARM64 NEON BSL 等目标指令。本人 x86 实测中, AVX2 在多种长度上取得约 6.28–8.56 倍加速; 支持 AVX-512 的验证环境中, AVX-512 在 4096 B 消息上取得约 12.89 倍加速。

综合来看, 本实验满足“ARM64 与 x86、AVX2 与 AVX-512、SIMD 与通用寄存器混合”的作业要求, 同时明确区分了源码可编译、目标指令存在、运行正确和性能实测四类证据。后续只需在真实 ARM64 设备上运行现成测试与基准命令, 即可补齐该平台的实测性能表。

A 编译、测试与跑分命令

A.1 x86-64

```
make clean
make -j"$(nproc)" all
make test
make verify-openssl
make asm-check
./build/sm3_bench 4096 256
make sanitize
```

A.2 ARM64 原生

```
make clean
make -j"$(nproc)" all
make test
make asm-check
./build/sm3_bench 4096 256
```

A.3 x86 主机交叉编译 ARM64

```
sudo apt install gcc-aarch64-linux-gnu binutils-aarch64-linux-gnu
make arm64
```

A.4 一键验收

```
TOTAL_MIB=256 ./scripts/run_validation.sh
```

B 关键公共 API

```
1 typedef enum sm3_backend {
2     SM3_BACKEND_SCALAR,
3     SM3_BACKEND_AVX2,
4     SM3_BACKEND_AVX512,
5     SM3_BACKEND_NEON,
6     SM3_BACKEND_AUTO
```

```
7 } sm3_backend;  
8  
9 void sm3_digest(const uint8_t *msg, size_t len, uint8_t out[32]);  
10  
11 int sm3_hash_many(sm3_backend backend,  
12                  const uint8_t *const *msg,  
13                  const size_t *len,  
14                  uint8_t (*out)[32],  
15                  size_t count);
```

参考文献

- [1] 国家质量监督检验检疫总局、国家标准化管理委员会. GB/T 32905-2016 信息安全技术 SM3 密码杂凑算法, 2016.
- [2] IETF. The SM3 Cryptographic Hash Function. Internet-Draft, algorithm description corresponding to GB/T 32905-2016.
- [3] Intel Corporation. Intel Intrinsics Guide and Intel 64 and IA-32 Architectures Software Developer's Manual.
- [4] GNU Project. GCC x86 Options: AVX2 and AVX-512 feature flags.
- [5] Arm Limited. Arm Neon Intrinsics Reference and Cortex-A Series Programmer's Guide for Armv8-A.
- [6] 山东大学网络空间安全学院. SM3 软件实现课程讲义, 2026.